

Agentic RAG

- Agent가 검색 전략 자율 결정

1. Agent RAG 구현

1. Agentic RAG란?

Agentic RAG는 RAG 시스템에 Agent의 판단 능력을 결합하여, 질문에 따라 검색 방법과 검색 횟수를 스스로 결정하도록 만든 구조이다.

일반적인 RAG에서는 검색 과정이 미리 정해져 있다.

사용자 질문



Vector Search



검색 결과



LLM



답변



검색 방법이나 횟수가 코드에 고정되어 있기 때문에 질문의 종류가 달라져도 동일한 검색 전략을 사용한다.

반면 Agentic RAG에서는 Agent가 질문과 검색 결과를 분석하고 다음 행동을 결정한다.

사용자 질문



Agent



검색할까?

└ 아니오 → 답변



└ 예



어떤 검색을 사용할까?

└ Vector Search

└ Graph Search



검색 결과 확인



충분한가?

└ 예 → 답변

└ 아니오 → 다른 검색 수행

즉, Agentic RAG에서는 검색 자체뿐만 아니라 "어떤 방식으로 검색할 것인가"도 Agent가 결정한다.

2. 왜 Agentic RAG가 필요한가?

질문마다 적절한 검색 전략이 다르기 때문이다.

예를 들어 다음과 같은 질문이 있다고 하자.

검색이 필요 없는 질문

"안녕하세요."

단순한 인사말이므로 검색할 필요가 없다.

단순 사실 질문

"이서연은 어느 팀 팀장이야?"

하나의 사실을 찾으면 되므로 Vector Search 한 번이면 충분하다.

관계형 질문

"김민준이 담당하는 프로젝트가 의존하는 프로젝트는 누가 담당해?"

여러 Entity와 관계를 연결해야 하므로 Graph Search가 적합하다.

복합 질문

"우리 회사 AI 관련 프로젝트를 담당하는 팀은 어떤 팀과 협업해?"

먼저 Vector Search를 통해 관련 정보를 찾고, 이후 Graph Search를 이용하여 Entity 간 관계를 확인하는 방식이 적합할 수 있다.

따라서 모든 질문에 동일한 검색 방법을 적용하는 것보다 질문의 특성에 따라 검색 전략을 변경하는 것이 효과적이다.

3. Agentic RAG의 핵심 구조

Agentic RAG에서는 검색 기능을 Agent가 사용할 수 있는 Tool로 구성한다.

대표적인 검색 Tool은 다음과 같다.

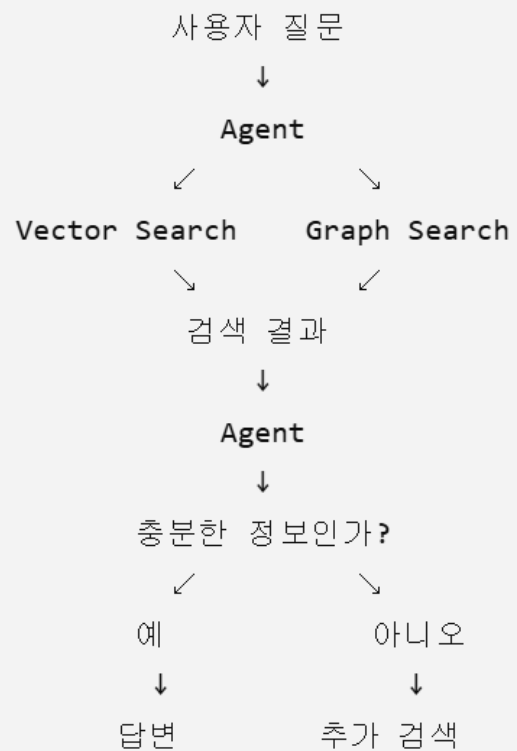
Vector Search

→ 의미적으로 유사한 문서를 검색

Graph Search

→ Entity 간 관계를 탐색

전체적인 구조는 다음과 같다.



여기서 핵심은 Agent가 검색 Tool을 직접 선택한다는 점이다.

4. Agent가 검색 전략을 결정하는 과정

Agentic RAG에서는 검색 전략을 미리 고정하지 않는다.

Agent는 다음과 같은 과정을 거쳐 행동을 결정한다.

① 검색이 필요한가?



② 어떤 **Tool**을 사용할 것인가?



③ 검색 결과가 충분한가?



④ 부족하다면 어떤 검색을 추가할 것인가?



⑤ 충분하면 검색을 종료하고 답변

예를 들어 단순한 질문이라면 검색을 생략할 수 있다.

질문



Agent



검색 불필요



답변

반대로 정보가 필요한 질문이라면 적절한 검색 Tool을 선택한다.

질문



Agent



Vector Search



결과 확인



충분 → 답변

검색 결과가 부족하면 다시 다른 Tool을 선택할 수 있다.

질문
↓
Agent
↓
Vector Search
↓
결과 부족
↓
Graph Search
↓
결과 확인
↓
답변

따라서 Agentic RAG에서는 검색 횟수와 검색 순서도 고정되어 있지 않다.

5. ReAct 기반 Agentic RAG

Agentic RAG는 Agent와 Tool이 반복적으로 상호작용하는 구조로 구현할 수 있다.

```
graph TD; Agent1[Agent] --> ToolCall1[Tool 호출]; ToolCall1 --> ToolResult1[Tool 결과]; ToolResult1 --> Agent2[Agent]; Agent2 --> ToolCall2[Tool 호출]; ToolCall2 --> ToolResult2[Tool 결과]; ToolResult2 --> Agent3[Agent]; Agent3 --> FinalAnswer[최종 답변];
```

Agent
↓
Tool 호출
↓
Tool 결과
↓
Agent
↓
Tool 호출
↓
Tool 결과
↓
Agent
↓
최종 답변

이를 ReAct 방식으로 표현하면 다음과 같다.

```
Think
↓
Act
↓
Observe
↓
Think
↓
Act
↓
Observe
↓
Answer
```

Agent는 검색 결과를 확인한 후 다음 행동을 다시 판단한다.

따라서 단순한

질문 → 검색 → 답변

구조가 아니라

```
질문
↓
판단
↓
검색
↓
결과 확인
↓
재판단
↓
추가 검색 또는 답변
```

과 같은 동적인 흐름을 갖는다.

6. 검색 전략의 전환

Agentic RAG의 중요한 특징은 검색 결과에 따라 전략을 변경할 수 있다는 것이다.

예를 들어 질문이 다음과 같다고 하자.

"우리 회사 AI 관련 프로젝트를 담당하는 팀은 어떤 팀과 협업해?"

Agent는 다음과 같이 행동할 수 있다.

질문

↓

Vector Search

↓

AI팀이 그래프 RAG 엔진 프로젝트 담당

↓

Entity = AI팀

↓

Graph Search

↓

AI팀 — 협업 — 데이터팀

↓

답변



처음부터 Graph Search만 사용하도록 고정하는 것이 아니라 첫 번째 검색 결과를 바탕으로 다음 검색 전략을 결정하는 것이다.

7. Multi-hop 질문에서의 Agentic RAG

Agentic RAG는 여러 단계의 검색이 필요한 질문에서도 활용할 수 있다.

다음 질문을 생각해 보자.

"김민준이 담당하는 프로젝트가 의존하는 프로젝트는 누가 담당해?"

필요한 정보는 다음과 같이 연결되어 있다.

김민준
↓
담당
↓
그래프 RAG 엔진
↓
의존
↓
검색 인프라 개선
↓
담당
↓
정다운

이 경우 한 번의 검색으로 모든 정보를 얻기 어려울 수 있다.

Agent는 첫 번째 검색 결과를 확인하고 새로운 Entity를 발견한 후 다시 검색할 수 있다.

1차 검색

김민준 → 그래프 RAG 엔진 → 검색 인프라 개선



2차 검색

검색 인프라 개선 → 정다운



최종 답변

따라서 검색 횟수 역시 질문에 따라 동적으로 결정된다.

8. Agentic RAG와 기존 RAG 비교

구분	Naive Vector RAG	Fixed Graph RAG	Agentic RAG
검색 여부	항상 검색	항상 검색	Agent가 결정
검색 전략	Vector 고정	Graph 고정	질문에 따라 선택
검색 횟수	고정	고정	가변
검색 실패	종료	종료	다른 전략으로 전환 가능
Multi-hop	상대적으로 약함	강함	필요할 때 Graph 사용
비용	낮음	낮음	상대적으로 높음
유연성	낮음	중간	높음

핵심적인 차이는 다음과 같다.

Fixed RAG

→ 검색 방법을 개발자가 결정

Agentic RAG

→ 검색 방법을 **Agent**가 결정

9. Agentic RAG의 장점

① 질문에 따라 검색 전략을 변경할 수 있다

단순 질문
→ 검색 생략

사실 조회
→ Vector Search

관계 질문
→ Graph Search

복합 질문
→ Vector + Graph

② 검색 횟수를 동적으로 결정한다

1회 검색
↓
충분
↓
답변

또는

1회 검색
↓
부족
↓
2회 검색
↓
충분
↓
답변

③ 검색 실패에 대응할 수 있다

Graph Search 실패



Vector Search



Entity 확인



Graph Search 재시도

④ 다양한 Tool로 확장할 수 있다

Agent에게 여러 종류의 Tool을 제공하면 검색 전략의 범위를 확장할 수 있다.

Vector Search

Graph Search

Web Search

SQL Query

API 호출

문서 검색

Agent는 질문의 특성에 따라 필요한 Tool을 선택할 수 있다.

10. Agentic RAG의 한계

Agentic RAG가 항상 기존 RAG보다 좋은 것은 아니다.

가장 큰 문제는 비용과 지연 시간이다.

고정된 RAG는 다음과 같이 단순하게 동작한다.

질문



검색



답변

반면 Agentic RAG에서는 여러 번의 판단과 Tool 호출이 발생할 수 있다.

질문
↓
Agent
↓
검색
↓
Agent
↓
검색
↓
Agent
↓
답변

검색 단계가 많아질수록 LLM 호출 횟수가 증가하므로 비용과 응답 시간이 증가한다.

따라서 질문 유형이 단순하고 일정한 서비스에서는 오히려 고정된 RAG가 더 효율적일 수 있다.

또한 Agent가 잘못된 검색 전략을 선택하거나 불필요하게 여러 번 검색하는 문제도 발생할 수 있다.

11. 핵심 정리

Agentic RAG의 핵심은 "검색을 잘하는 RAG"가 아니라 "어떻게 검색할 것인지 스스로 결정하는 RAG"이다.

일반적인 RAG는 검색 전략이 고정되어 있다.

일반 RAG



질문



정해진 검색



답변

Agentic RAG는 검색 과정 자체가 동적으로 결정된다.

Agentic RAG

질문



Agent



검색 전략 결정



Tool 실행



결과 확인



추가 검색 또는 종료



답변

"Graph RAG는 관계를 이용해 검색하고, Agentic RAG는 어떤 검색 전략을 사용할지를 Agent가 결정한다."

마지막으로 네 가지 고급 RAG 기법의 차이는 다음과 같이 정리할 수 있다.

Graph RAG

→ 관계 기반 검색

Agentic RAG

→ 검색 전략을 자율 결정

Self-Corrective RAG

→ 검색 결과를 평가하고 필요하면 재검색

RAG Fusion

→ 여러 검색 결과를 결합



즉, Graph RAG는 "관계", Agentic RAG는 "판단", Self-Corrective RAG는 "검증", RAG Fusion은 "통합"에 초점을 둔 기술이다.



감사합니다